

FOR CLAUDE CODE USERS

# 18% Faster. 3.6× Sooner.

Opus 5.5, tested in Claude Code

The raw speed gain was modest. The big difference appeared once the model became an agent.

Three claims from the launch page, tested with a harness anyone can rerun: a generation test with no tools, and three real coding jobs graded by tests the models never saw.

Opus 5.5 against Opus 5 on everything, and against Fable 5.1 on the coding jobs. Claude Code, headless, effort never set.

Models, prices and defaults change. Everything on this sheet was true on 27 September 2026. The method is the durable part: fix the tasks, hide the tests, count tokens by category, and rerun it on the day you decide.

## In scope

- Generation speed on one fixed prompt, ten runs per model
- Three backend coding jobs, three runs per model, held-out tests
- Tokens by category, priced at published API rates

## Out of scope

- Anthropic's "typical workloads" mix, which was not reproduced
- Quality beyond pass or fail: every model passed every check

Three runs per job show the spread, not statistical significance. Every run is in the repository.

## 01 The findings, in one table

|                                         |                                                     |
|-----------------------------------------|-----------------------------------------------------|
| GENERATION: text tokens per second      | 0pus 5.5 159.9, 0pus 5 136.0: +18%                  |
| GENERATION: thinking included           | 161.4 against 127.8: +26%                           |
| GENERATION: time to first text          | 3.45 s against 11.12 s                              |
| AGENTIC JOBS: total wall time, 9 runs   | 0pus 5.5 1,341 s, 0pus 5 4,823 s, Fable 5.1 2,991 s |
| AGENTIC JOBS: API list price equivalent | \$4.91, \$16.47, \$20.18: 70% below 0pus 5          |
| QUALITY CHECKS: every check passed      | 9 of 9 for all three models: a ceiling              |

Generation, agentic jobs and quality checks are separate measurements.

## 02 The claims, verbatim

"generates output more than 30% faster than Opus 5". "at default settings it will cost 40% less than Opus 5 on typical workloads". "performs at the level of Claude Fable 5.1 on most work".

Prices per million tokens at the time of recording: Opus 5.5 \$4 in, \$20 out, \$0.20 cache read; Opus 5 \$5, \$25, \$0.50; Fable 5.1 \$10, \$50, \$0.25.

The three claims, as quoted.

Introducing Claude Opus 5.5, Anthropic, 22 September 2026, read 27 September 2026

The per-million rates.

Anthropic pricing page, read 27 September 2026

## 03 Why 40% is not 20%

The list price drops 20%, but an agent sends the conversation back every turn, and most of it is a cache read, which drops 60%. So the saving is never guessed from the price table: every token is counted by category and priced on its own.

```
harness/run_tasks.py: the cost of one run
```

```
def price(model_id: str, uncached: int, w5m: int, w1h: int, read: int, out: int) → float | None:
    r = rates_for(model_id)
    if r is None:
        return None
    return (uncached * r["in"] + w5m * r["w5m"] + w1h * r["w1h"] + read * r["read"] + out * r["out"]) / 1e6
```

```
One real run: Opus 5.5, Kafka job, run 1
```

```
uncached input      26 x $4
cache writes        34,299 x $8
cache reads         276,575 x $0.20
output              18,087 x $20
                    per million = $0.69
```

The calculated cost matched Claude Code's own `total_cost_usd` to the cent on all 27 runs.

Run in Claude Code on 2026-09-27. Reproduce it with the harness in the repository.



## 04 Making the comparison fair

Each job comes from a frozen commit of a public repository, renamed so no model can recall the published code. The same prompt, the same Bash allowlist and the same checks for every model; who goes first rotates; effort is never set. The checks run after the model has finished, on a copy it never saw, and each discriminating check was proven to fail on a deliberately wrong solution first.

T2: every statement prepared on a JDBC connection is counted

```
int statements(Runnable r) {
    STATEMENTS.set(0);
    COUNTING.set(true);
    try {
        r.run();
    } finally {
        COUNTING.set(false);
    }
    return STATEMENTS.get();
}
```

A one-query-per-customer count ran 34 statements against a limit of 3; Spring's default Kafka error handler made 10 attempts where 4 and 1 were required. Both failed exactly the check meant to catch them.

Run in Claude Code on 2026-09-27. Reproduce it with the harness in the repository.

## 05 The one correction

One run first graded 7 of 8: the SQL check rejected any use of `Sprintf`, even where it only numbered a placeholder and the value was still bound. The check was corrected once and every saved solution was re-graded. Only that grade changed.

tasks/T1/checks/param\_check.py: the corrected rule

```
def ok(body: str) → bool:
    m = re.search(r"func \(s \*Store\) List\(\s*\w+ \[w.\]+\s*(\w+) string", body)
    if not m or "$1" not in body:
        return False
    v = m.group(1)
    code = re.sub(r'`[^`]*`', '``', body, flags=re.S)
    code = re.sub(r'"(\\"|\\.|[^\\"\\])*"', '""', code)
    code = re.sub(r"//[^\n]*", "", code)
    for call in re.finditer(r"Sprintf\(", code):
        depth, i = 1, call.end()
        while i < len(code) and depth:
            depth += {"(": 1, ")": -1}.get(code[i], 0); i += 1
        if re.search(rf"\b{v}\b", code[call.end():i]):
            return False
    return not re.search(rf"\s*\{v}\b|\b{v}\s*", code)
```



## 06 What this does not show

It does not show the 30% claim is false: Anthropic does not publish its method. It does not verify the 40%: that is a typical mix, and these are three backend jobs. It does not verify Fable level quality: the tests hit a ceiling. Opus 5.5 is not always 70% cheaper or 3.6 times faster, and none of this says Fable is not worth it.

## 07 The questions this answers

Why can a model be 18% faster at generating and 3.6 times faster at finishing a job?

Why is an agent's bill mostly cache reads?

What does a test prove when every model passes it?

Why time the first word separately from tokens per second?

How do you keep a model from fitting its answer to your tests?

### Before you trust a model comparison

1. The tasks are fixed, and the model cannot have seen the solutions
2. The tests are hidden from the model and proven to fail on a wrong answer
3. Generation speed, finished-job time and quality are measured separately
4. Cost is counted by token category, not guessed from the list price
5. Every run is kept, and the spread is shown, not only the median
6. Any correction to a checker is disclosed and applied to every run

## Everything else

- |                                                                                                                                                                                                                          |                                                                                                                                                                                                     |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  <b>Site</b><br><a href="https://code-with-sam-dev.github.io">https://code-with-sam-dev.github.io</a>                                   |  <b>Facebook</b><br><a href="https://www.facebook.com/codewithsamdev">https://www.facebook.com/codewithsamdev</a>  |
|  <b>YouTube</b><br><a href="https://www.youtube.com/@CodewithSam-Dev">https://www.youtube.com/@CodewithSam-Dev</a>                      |  <b>GitHub</b><br><a href="https://github.com/code-with-sam-dev">https://github.com/code-with-sam-dev</a>          |
|  <b>LinkedIn</b><br><a href="https://www.linkedin.com/company/code-with-sam-dev">https://www.linkedin.com/company/code-with-sam-dev</a> |  <b>Buy me a coffee</b><br><a href="https://buymeacoffee.com/codewithsam">https://buymeacoffee.com/codewithsam</a> |
|  <b>x</b><br><a href="https://x.com/CodeWithSamDev">https://x.com/CodeWithSamDev</a>                                                    |  <b>Buy me sadza</b><br><a href="https://buymesadza.com/codewithsam">https://buymesadza.com/codewithsam</a>        |
|  <b>TikTok</b><br><a href="https://www.tiktok.com/@codewithsamdev">https://www.tiktok.com/@codewithsamdev</a>                           | <b>The test harness and every raw run</b><br><a href="https://github.com/code-with-sam-dev/claude-code-opus-test">https://github.com/code-with-sam-dev/claude-code-opus-test</a>                    |
|  <b>Instagram</b><br><a href="https://www.instagram.com/codewithsamdev">https://www.instagram.com/codewithsamdev</a>                    | <b>Anthropic pricing</b><br><a href="https://platform.claude.com/docs/en/about-claude/pricing">https://platform.claude.com/docs/en/about-claude/pricing</a>                                         |

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which part of your workflow you want taken apart next.

Claude, Claude Code, Opus and Fable are trademarks of Anthropic. Go is a trademark of Google LLC. Spring is a trademark of Broadcom. This is an independent, unofficial test produced by Code with Sam, not affiliated with or endorsed by any of them, and no third party artwork is reproduced here.

Technical claims verified 2026-09-27 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.